内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

【数据结构】面试题整理 共36题

weixin_44329069 已于 2024-06-21 17:22:30 修改

面试系列 专栏收录该内容

3 订阅 18 篇文章 订阅专栏

文章目录

1. 简述什么是数据结构?
2. 常见的数据结构有哪些?
3. 简述什么是链表?
4. 简述链表的分类?
5. 简述链表与数组的区别?
6. 链表的应用场景有哪些?
7. 简述什么是栈?
8. 说一说栈有哪些应用场景?
9. 栈的内存是怎么分配的?
10. 栈溢出的原因以及解决方法?
11. 简述什么是队列?
12. 简述队列的使用场景?
13. 叙述栈和队列的区别?
14. 简述什么是堆?
16. 说一说堆有哪些应用场景?
17. 简述堆和普通树的区别?
18. 简述堆和栈的区别?
19. 数据结构中头指针和头结点的区别?
20. 简述什么是哈希表?
21. 哈希表冲突的解决办法有哪些?
22. 哈希表有哪些优缺点?
23. 什么情况下可是实用哈希表?
24. 简述什么是中缀、前缀、后缀符号?

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

为什么使用不同的表示法?
25. 简述什么是排序二叉树?
26. 简述什么是前缀树?
27. 什么是平衡二叉树?
28. 平衡二叉树有哪些优缺点?
优点
缺点
29. 简述什么是红黑树?
30. 红黑树适合什么样的使用场景?
31. 平衡二叉树和红黑树有什么区别?
32. 简述什么是满二叉树?
33. 简述什么是完全二叉树?
特点
应用
与满二叉树和平衡二叉树的关系
34. 简述二叉树的存储方式?
35. 简述什么是B-tree、B+tree多叉树?
B-tree（B树）
B+tree（B+树）
B树和B+树的比较
36. 综合简述B 树和B+ 树的区别?

---

1. 简述什么是数据结构?

数据结构就是一种组织和存储数据的方式，使得我们可以高效地访问和修改数据。就像你整理房间一样，不同的物品有不同的收纳方式，数据也有不同的存储和管理方法。

比如，数组就像一个排成一排的抽屉，每个抽屉都有编号，你可以很快找到特定编号的抽屉里的东西。链表则像一串珍珠项链，每颗珍珠（节点）都知道下一颗珍珠在哪里。还有像树结构，它就像家谱一样，每个人都有父母和子女，层层分明。

这些不同的数据结构各有优缺点，适用于不同的场景。选择合适的数据结构可以让你的程序运行得更快、更高效。

内容来源：csdn.net
作者昵称：weixin_44329069
文章链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

2. 常见的数据结构有哪些?

常见的数据结构有很多种，每种都有其独特的特点和适用场景。以下是一些常见的数据结构：

1. **数组（Array）**：一种线性结构，元素在内存中连续存储，访问速度快，但插入和删除操作较慢。
2. **链表（Linked List）**：元素通过指针连接，插入和删除操作方便，但访问速度较慢。
3. **栈（Stack）**：一种后进先出（LIFO）的数据结构，常用于递归和回溯操作。
4. **队列（Queue）**：一种先进先出（FIFO）的数据结构，常用于任务调度和广度优先搜索。
5. **哈希表（Hash Table）**：通过哈希函数将键映射到数组中的位置，查找速度快，但需要处理哈希冲突。
6. **树（Tree）**：一种层次结构，常用于表示具有层次关系的数据，如文件系统。二叉树、红黑树和B+树是常见的树结构。
7. **堆（Heap）**：一种特殊的树结构，用于实现优先队列，常用于排序算法。
8. **图（Graph）**：由节点和边组成，用于表示复杂的关系网络，如社交网络和交通网络。

这些数据结构各有优缺点，选择合适的数据结构可以显著提高程序的效率和性能。

3. 简述什么是链表?

链表是一种常见的数据结构，它由一系列节点组成，每个节点包含数据和一个指向下一个节点的指针。与数组不同，链表中的元素在内存中不必是连续存储的，这使得链表在插入和删除操作时非常高效。

链表有几种常见类型：

1. **单向链表（Singly Linked List）**：每个节点只包含一个指向下一个节点的指针。
2. **双向链表（Doubly Linked List）**：每个节点包含两个指针，一个指向下一个节点，一个指向前一个节点。
3. **循环链表（Circular Linked List）**：链表的最后一个节点指向第一个节点，形成一个环。

链表的优点是插入和删除操作非常高效，特别是在需要频繁插入和删除的场景下。缺点是访问特定位置的元素时速度较慢，因为需要从头开始逐个遍历。

作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

4. 简述链表的分类?

链表有几种常见的分类，每种都有其独特的特点和用途：

1. **单向链表（Singly Linked List）**：每个节点包含数据和一个指向下一个节点的指针。它的优点是结构简单，插入和删除操作高效，但只能从头到尾遍历。
2. **双向链表（Doubly Linked List）**：每个节点包含数据、一个指向下一个节点的指针和一个指向前一个节点的指针。双向链表可以从头到尾和从尾到头遍历，插入和删除操作也很方便，但需要更多的内存来存储额外的指针。
3. **循环链表（Circular Linked List）**：链表的最后一个节点指向第一个节点，形成一个环。循环链表可以是单向的也可以是双向的，适用于需要循环访问的场景。
4. **双向循环链表（Doubly Circular Linked List）**：结合了双向链表和循环链表的特点，既可以双向遍历，又形成一个环，适用于需要频繁遍历和循环访问的场景。

这些链表类型各有优缺点，选择合适的链表类型可以根据具体的应用场景来决定。

5. 简述链表与数组的区别?

链表和数组是两种常见的数据结构，它们在存储方式和操作效率上有显著的区别：

1. **存储方式**：
   - **数组**：在内存中是连续存储的，每个元素占据固定的内存空间。数组的大小在定义时就固定了，不能动态改变。
   - **链表**：在内存中是非连续存储的，每个节点包含数据和指向下一个节点的指针。链表的大小可以动态改变，节点可以在运行时动态分配和释放。
2. **访问效率**：
   - **数组**：由于是连续存储，可以通过下标快速访问任意元素，访问时间复杂度为 $O(1)$。
   - **链表**：需要从头开始逐个遍历，访问某个特定元素的时间复杂度为 $O(n)$。
3. **插入和删除操作**：
   - **数组**：在中间插入或删除元素时，需要移动其他元素，时间复杂度为 $O(n)$。
   - **链表**：只需修改相关节点的指针，不需要移动其他元素，插入和删除操作的时间复杂度为 $O(1)$。
4. **内存利用**：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

- **数组**：由于大小固定，可能会造成内存浪费或溢出。
- **链表**：动态分配内存，内存利用率高，但需要额外的指针存储空间。

5. **适用场景**：
   - **数组**：适用于需要频繁访问元素的场景，如查找操作。
   - **链表**：适用于需要频繁插入和删除元素的场景，如队列和栈的实现。

6. 链表的应用场景有哪些?

链表在很多场景中都非常有用，特别是在需要频繁插入和删除操作的情况下。以下是一些常见的应用场景：

1. **动态数据结构**：链表非常适合用于需要频繁插入和删除元素的数据结构，因为它们不需要移动其他元素，只需调整指针即可。
2. **内存管理**：链表可以用于实现自定义的内存管理器，动态分配和释放内存块。这在操作系统和嵌入式系统中非常常见。
3. **事件处理**：在事件驱动的系统中，链表可以用于存储待处理的事件，按照事件发生的顺序或优先级进行处理。
4. **深度优先搜索（DFS）**：在图形遍历算法中，链表常用于表示节点的邻接关系，从而实现递归遍历。
5. **LRU缓存**：链表可以用于实现最近最少使用（LRU）缓存算法，帮助管理缓存中的数据，确保最近使用的数据优先保留。

7. 简述什么是栈?

栈是一种特殊的线性数据结构，只允许在一端进行插入和删除操作。这一端被称为栈顶，另一端称为栈底。栈遵循“后进先出”（LIFO，Last In First Out）的原则。

简单来说，栈就像一叠盘子，你只能从顶部添加或移除盘子。最新放上去的盘子会最先被拿走。

栈的主要操作有：

- **压栈（Push）**：将一个元素添加到栈顶。
- **出栈（Pop）**：从栈顶移除一个元素。
- **查看栈顶元素（Peek/Top）**：获取栈顶的元素但不移除它。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

栈在计算机科学中有很多应用，比如函数调用的管理、表达式求值、括号匹配等。

8. 说一说栈有哪些应用场景?

栈是一种非常实用的数据结构，它的后进先出（LIFO）特性让它在多种编程和系统设计场景中有着广泛的应用：

1. **函数调用管理**：在程序执行过程中，栈用于管理函数调用。每当一个函数被调用时，当前的执行状态（包括局部变量和返回地址）会被压入栈中。当函数返回时，这些信息会从栈中弹出并恢复[1]。
2. **表达式求值**：栈可以用于计算表达式的值，特别是在将中缀表达式转换为后缀表达式（逆波兰表达式）时。
3. **括号匹配**：编译器和解释器使用栈来检查括号是否匹配。每当遇到一个左括号时，将其压入栈中；每当遇到一个右括号时，从栈中弹出一个左括号进行匹配。
4. **撤销操作**：许多软件应用程序（如文本编辑器）使用栈来实现撤销功能。每当用户执行一个操作时，相关信息会被推入栈中。当用户选择撤销时，程序会从栈中弹出最近的操作并还原到上一个状态。
5. **浏览器历史记录**：浏览器使用栈来实现后退和前进功能。每当访问一个新页面时，当前页面的信息会被推入栈中。当用户点击后退按钮时，程序会从栈中弹出最近访问的页面并显示上一个页面。
6. **递归算法**：递归函数调用本质上使用栈来保存每次递归调用的状态。当递归函数返回时，栈会弹出并恢复上一个状态。

9. 栈的内存是怎么分配的?

栈的内存分配方式依赖于其在计算机系统中的具体实现。在大多数现代操作系统和编程环境中，栈主要用于两个目的：一是作为数据结构在程序代码中显式使用；二是作为程序执行栈（调用栈）隐式使用。两者在内存分配上有所不同：

1. 数据结构中的栈
   当栈被用作数据结构时（例如，程序员在代码中显式创建和使用的栈），其内存分配方式取决于栈的实现（数组或链表）和所用编程语言的内存管理机制。

- 基于数组的栈：
  这种栈通常在数组初始化时分配一块连续的内存空间，数组的大小可能是固定的，也可能是动态扩展的（如Java的ArrayList或C++的std::vector）。动态扩展可能涉及到在栈增长到超过当前容量时，分配一个更大的内存块，将旧元素复制到新位置，并释放原始内存。
- 基于链表的栈：
  链表实现的栈在每次添加新元素时动态分配内存（每个节点一个），通常使用堆内存。每个节点包含数据和指向下一个节点的指针，不需要连续的内存空间。

来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

2. 程序执行栈（调用栈）
   程序执行栈是操作系统为每个线程自动创建的一块内存区域，用于存储函数调用的上下文（包括返回地址、局部变量、参数等）。这种栈的特点是：

- 固定大小：操作系统为每个线程分配的调用栈大小通常是固定的，其大小在程序启动时确定，但可以在某些操作系统和编程环境中配置。如果一个程序超出这个大小限制（栈溢出），会导致程序崩溃或不可预期的行为。
- 自动管理：程序员通常不直接管理调用栈的内存分配和回收，这些都由编译器和操作系统自动处理。函数调用时，相关上下文自动“推入”栈中；函数返回时，上下文自动“弹出”栈，返回地址被用来恢复执行流。

栈的内存分配方式既可以是静态的（如基于数组的实现，预先分配固定大小的内存），也可以是动态的（如基于链表的实现，按需分配内存）。而对于程序的执行栈，其内存通常是由操作系统在程序启动时自动分配的固定大小空间，专门用于处理函数调用的上下文。

10. 栈溢出的原因以及解决方法?

栈溢出通常是因为程序在运行时，栈内存被耗尽了。常见的原因有以下几种：

1. **递归调用太深**：如果一个函数不断调用自己而没有终止条件，就会导致栈溢出。
2. **局部变量过多**：在一个函数中定义了太多的局部变量，也会占用大量的栈空间。
3. **方法调用链太长**：如果一个方法调用了很多其他方法，尤其是在循环中，也可能导致栈溢出。

解决方法有：

1. **优化递归算法**：确保递归有明确的终止条件，或者使用循环代替递归。
2. **减少局部变量**：尽量减少函数中的局部变量数量，或者将一些变量转换为成员变量。
3. **增加栈的大小**：可以通过调整虚拟机参数来增加栈的大小，比如在Java中使用`-Xss`参数。

11. 简述什么是队列?

队列是一种常见的数据结构，它按照**先进先出**（FIFO, First In First Out）的原则进行操作。简单来说，队列就像排队买票，先到的人先买票，后到的人排在队尾。

在队列中，有两个主要操作：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

1. **入队**：将元素添加到队列的末尾，称为队尾。
2. **出队**：从队列的开头移除元素，称为队头。

队列的应用非常广泛，比如打印任务的管理、进程调度等。

12. 简述队列的使用场景?

队列在计算机科学和日常生活中有很多应用场景。以下是一些常见的使用场景：

1. **任务调度**：操作系统中使用队列来管理任务的执行顺序，确保任务按照先来先服务的原则进行处理。
2. **缓冲区管理**：在网络通信中，队列用于数据包的缓冲，确保数据按顺序传输和处理。
3. **打印队列**：打印机处理多个打印任务时，会将任务按顺序排队，先提交的任务先打印。
4. **广度优先搜索**：在图算法中，队列用于实现广度优先搜索（BFS），帮助遍历图中的节点。
5. **消息队列**：在分布式系统中，消息队列用于异步处理任务，解耦系统组件，提高系统的可扩展性和可靠性。
6. **物流与供应链**：在生产线和物流中，队列用于顺序安排作业任务或货物处理。

13. 叙述栈和队列的区别?

栈和队列是两种常见的数据结构，它们在操作方式和应用场景上有明显的区别：

1. **操作规则**：
   - **栈**：遵循后进先出（LIFO, Last In First Out）的原则。也就是说，最后一个放入栈的元素最先被取出。想象一下子弹夹，最后压入的子弹最先被射出。
   - **队列**：遵循先进先出（FIFO, First In First Out）的原则。也就是说，第一个进入队列的元素最先被取出。就像排队买票，先到的人先买票。
2. **操作位置**：
   - **栈**：插入和删除操作都在栈顶进行。你只能从栈顶添加或移除元素。
   - **队列**：插入操作在队尾进行，删除操作在队头进行。你从一端添加元素，从另一端移除元素。
3. **应用场景**：
   - **栈**：常用于实现递归算法、表达式求值、函数调用管理等。例如，浏览器的前进和后退功能就是用栈来实现的。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
版权声明：https://blog.csdn.net/weixin_44329069

- **队列**：常用于任务调度、广度优先搜索（BFS）、消息队列等。例如，打印任务管理和操作系统的任务调度都使用队列。

14. 简述什么是堆?

堆（Heap）是一种特殊的 树形数据结构，通常用于实现优先队列。堆有以下几个特点：

1. **完全二叉树**：堆是一棵完全二叉树，这意味着除了最后一层，所有层都是满的，最后一层的节点从左到右依次排列。
2. **堆性质**：在堆中，每个节点的值都不大于（最大堆）或不小于（最小堆）其子节点的值。这保证了堆顶（根节点）是整个堆中最大或最小的元素。

堆的主要操作有：

- **插入**：将新元素添加到堆中，并调整堆以保持堆性质。
- **删除**：通常删除堆顶元素，并用最后一个元素替代堆顶，然后调整堆以保持堆性质。

堆的常见应用包括：

- **优先队列**：堆可以高效地实现优先队列，支持快速插入和删除最大或最小元素。
- **堆排序**：一种基于堆的数据排序算法，时间复杂度为 $O(n \log n)$。
- **图算法**：如Dijkstra算法和Prim算法中，堆用于高效地选择最小权重边

堆通常通过数组实现。由于堆是完全二叉树，所以可以使用数组的索引来表示父子关系，即对于数组中任意位置i的元素，其左子节点的位置是2i+1，右子节点的位置是2i+2，父节点的位置是(i-1)/2（向下取整）。总结来说，堆是一种高效的数据结构，特别适用于需要快速访问和删除最大元素或最小元素的场景。

16. 说一说堆有哪些应用场景?

1. **优先级队列**：堆常用于实现优先级队列。比如在操作系统中，任务调度器会根据任务的优先级来决定哪个任务先执行。Java中的 `PriorityQueue` 就是用堆来实现的。
2. **求Top K问题**：当你需要找出一组数据中最大的前K个数或最小的前K个数时，可以使用堆来解决。比如在大数据分析 中，找出访问量最大的前K个网页。
3. **实时中位数计算**：在数据流中实时计算中位数，可以使用两个堆，一个大顶堆和一个小顶堆。大顶堆存储较小的一半数据，小顶堆存储较大的一半数据，这样堆顶元素就是中位数。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

4. **合并有序文件**：当你有多个有序的小文件需要合并成一个有序的大文件时，可以使用堆。将每个小文件的第一个元素放入堆中，然后依次取出堆顶元素并插入到大文件中，再将取出元素所在文件的下一个元素放入堆中，重复这个过程直到所有文件都合并完成。

17. 简述堆和普通树的区别?

堆和普通树在结构和用途上有一些显著的区别：

1. **结构**：
   - **堆**：堆是一种特殊的完全二叉树，分为最大堆和最小堆。在最大堆中，每个父节点的值都大于或等于其子节点的值；在最小堆中，每个父节点的值都小于或等于其子节点的值。
   - **普通树**：普通树可以是任意结构的树，不一定是完全二叉树。二叉搜索树（BST）是一种常见的普通树，其中左子节点的值小于父节点，右子节点的值大于父节点。
2. **内存占用**：
   - **堆**：堆通常使用数组来存储数据，不需要额外的指针来表示节点之间的关系，因此内存利用率较高。
   - **普通树**：普通树通常使用节点和指针来表示结构，每个节点需要额外的内存来存储指针。
3. **平衡性**：
   - **堆**：堆不需要严格的平衡，只需要满足堆的性质即可。插入和删除操作的时间复杂度为 $O(\log n)$。
   - **普通树**：二叉搜索树在最坏情况下可能会退化成链表，因此需要平衡树（如AVL树或红黑树）来保证操作的时间复杂度为 $O(\log n)$。
4. **应用场景**：
   - **堆**：常用于实现优先级队列、求Top K问题、实时中位数计算等。
   - **普通树**：广泛应用于数据库索引、文件系统、表达式解析等。

18. 简述堆和栈的区别?

堆和栈在计算机科学中有不同的用途和特性，以下是它们的主要区别：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

1. **内存管理**：

- **栈**：由操作系统自动分配和释放，用于存储函数的参数、局部变量等。栈的内存分配是连续的，效率较高。
- **堆**：由程序员手动分配和释放，用于动态分配内存。堆的内存分配是不连续的，可能会产生内存碎片，效率相对较低。

2. **生长方向**：
   - **栈**：内存地址从高到低增长，即栈顶向下扩展。
   - **堆**：内存地址从低到高增长，即堆顶向上扩展。
3. **分配方式**：
   - **栈**：分配方式简单，通常在函数调用时分配，函数返回时释放。栈的分配和释放速度非常快。
   - **堆**：分配方式复杂，需要程序员手动管理内存的分配和释放，容易产生内存泄漏。
4. **空间大小**：
   - **栈**：空间较小，通常由操作系统预先设定，大小有限。
   - **堆**：空间较大，理论上可以使用整个虚拟内存空间。
5. **用途**：
   - **栈**：适用于存储临时变量、函数调用等需要快速分配和释放的场景。
   - **堆**：适用于需要动态分配内存的场景，如对象的创建和销毁。

19. 数据结构中头指针和头结点的区别?

头指针和头结点在链表数据结构中有不同的作用和特性：

1. **头指针**：
   - **定义**：头指针是指向链表第一个节点的指针变量。它的作用是记录链表的起始地址，方便对链表的操作。
   - **作用**：头指针用于标识链表的起点，无论链表是否为空，头指针都存在。它指向链表的第一个节点（如果有头结点，则指向头结点；如果没有头结点，则指向第一个数据节点）。
2. **头结点**：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

- **定义**：头结点是链表中的一个特殊节点，通常不存储实际数据。它放在链表的最前面，主要用于简化链表的操作。
- **作用**：头结点的存在使得链表的插入和删除操作更加统一和方便。比如，在有头结点的链表中，插入和删除第一个数据节点的操作与其他节点的操作一致。

总结：

- **头指针**：指向链表的第一个节点，起标识作用。
- **头结点**：一个不存储实际数据的节点，用于简化链表操作。

20. 简述什么是哈希表?

哈希表（Hash Table）是一种用于存储键值对的数据结构。它通过一个称为哈希函数的函数，将键（Key）映射到表中的一个位置，从而加快查找速度。以下是哈希表的一些关键点：

1. **哈希函数**：哈希函数将键转换为哈希值，这个哈希值对应于哈希表中的一个位置。理想情况下，不同的键会映射到不同的位置。
2. **冲突处理**：由于哈希函数可能会将不同的键映射到相同的位置（称为冲突），因此需要有方法来处理这些冲突。常见的冲突处理方法包括链地址法（拉链法）和开放地址法[1,2].
3. **查找效率**：哈希表的查找、插入和删除操作在平均情况下都可以达到常数时间复杂度 $O(1)$，这使得它在需要快速查找的场景中非常有用。
4. **应用场景**：哈希表广泛应用于数据库索引、缓存实现、唯一性检测等场景。例如，在编译器中，哈希表用于存储符号表，以便快速查找变量和函数名。

21. 哈希表冲突的解决办法有哪些?

在哈希表中，当两个或多个键通过哈希函数映射到同一个位置时，就会发生冲突（Collision）。处理这种冲突的方法主要有以下几种：

1. **开放定址法**：当发生冲突时，寻找下一个空闲位置。具体方法包括：
   - **线性探测法**：如果位置被占用，就往后一个一个找，直到找到空位。
   - **平方探测法**：类似线性探测，但每次跳跃的步长是平方数，比如1, 4, 9等。
2. **再哈希法**：使用多个不同的哈希函数。如果第一个哈希函数发生冲突，就用第二个，依此类推，直到找到不冲突的位置。
3. **链地址法**：将所有哈希地址相同的元素存储在一个链表中。这样即使发生冲突，也可以通过链表来存储多个元素。

内容来源：csdn.net
作者：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
版权声明：https://blog.csdn.net/weixin_44329069

4. **建立公共溢出区**：将哈希表分为基本表和溢出表，发生冲突的元素都存放在溢出表中。

22. 哈希表有哪些优缺点?

哈希表有很多优点，但也有一些缺点。简单来说：

优点：

1. **快速查找**：哈希表的查找时间复杂度平均为O(1)，非常高效。
2. **插入和删除速度快**：同样由于哈希函数的作用，插入和删除操作也很快。
3. **实现简单**：哈希表的基本原理和实现相对简单，适合用于各种应用场景。

缺点：

1. **空间浪费**：为了减少冲突，哈希表通常需要预留较大的空间，这可能导致内存浪费。
2. **冲突处理复杂**：虽然有多种解决冲突的方法，但处理冲突仍然会增加复杂性和时间开销。
3. **哈希函数设计难**：设计一个好的哈希函数不容易，需要确保哈希值均匀分布，避免大量冲突。

23. 什么情况下可是实用哈希表?

哈希表在很多情况下都非常实用，尤其是需要快速查找、插入和删除操作的时候。以下是一些常见的应用场景：

1. **数据库索引**：哈希表可以用来实现数据库的索引，加快数据的查询速度。
2. **缓存**：在缓存系统中，哈希表可以快速存取数据，提升系统性能。
3. **字典和集合**：编程语言中的字典（如Python中的 `dict` ）和集合（如Python中的 `set` ）通常都是用哈希表实现的。
4. **计数**：在需要统计元素出现次数的场景中，哈希表可以高效地记录和查找每个元素的计数。
5. **去重**：哈希表可以用来快速判断一个元素是否已经存在，从而实现去重功能。

24. 简述什么是中缀、前缀、后缀符号?

中缀、前缀和后缀符号是三种不同的表达式表示方法，主要区别在于运算符的位置：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

1. **中缀表达式**：运算符位于操作数之间，中缀表达式需要考虑运算符的优先级和括号，以确定操作数的组合方式。这是我们最常见的表示方法。例如，$1+2$。
2. **前缀表达式**（波兰表示法）：运算符位于操作数之前，这种格式不需要括号来指示运算的顺序，运算的顺序遵循自顶向下的方式。例如，$+12$。
3. **后缀表达式**（逆波兰表示法）：运算符位于操作数之后，后缀表达式的求值可以从左向右进行，每遇到一个运算符就应用到之前的两个操作数上，适合计算机处理。例如，$12+$。

为什么使用不同的表示法?

可读性：中缀表示法对人类来说最直观易懂，因为它符合我们平时书写和阅读数学表达式的习惯。
易于解析：前缀和后缀表示法对计算机解析和计算来说更直接高效，因为它们避免了对运算符优先级和括号的处理。特别是在编写编译器和解释器时，利用栈结构可以简单地实现对前缀或后缀表达式的求值。

25. 简述什么是排序二叉树?

排序二叉树，也叫二叉搜索树（Binary Search Tree，简称BST），是一种特殊的二叉树。它有以下几个特点：

1. **每个节点的左子树**：所有节点的值都小于该节点的值。
2. **每个节点的右子树**：所有节点的值都大于该节点的值。
3. **左右子树**：每个子树本身也是一个二叉搜索树[1][2][3]。

这种结构使得查找、插入和删除操作都比较高效，平均时间复杂度为$O(\log n)$。

26. 简述什么是前缀树?

前缀树，也叫做字典树或Trie树，是一种用于存储字符串的特殊树形结构。它的主要特点是利用字符串的公共前缀来组织数据，从而提高查询效率。

简单来说，前缀树的每个节点代表一个字符，从根节点到某个节点的路径就构成了一个字符串。例如，如果我们有单词 “apple” 和 “app”，它们在前缀树中会共享 “app” 这个前缀部分，然后再分别延伸出 “le” 和空节点。

前缀树的主要优点是可以快速地进行字符串查找和前缀匹配，非常适合用于自动补全、拼写检查等应用场景。

27. 什么是平衡二叉树?

平衡二叉树，也叫做AVL树，是一种特殊的二叉搜索树。它的特点是任意节点的左子树和右子树的高度差不超过1。这样可以确保树的高度保持在较低水平，从而提高查找、插入和删除操作的效率。

内容来源：csdn.net
作者昵称：weixin_44329069
作者主页：https://blog.csdn.net/weixin_44329069

简单来说，平衡二叉树通过旋转操作来保持平衡。当插入或删除节点导致树不平衡时，会进行相应的旋转操作来恢复平衡。这些旋转操作包括左旋、右旋、左右旋和右左旋。

28. 平衡二叉树有哪些优缺点?

优点

1. **高效的查找、插入和删除操作**：由于树的高度保持在对数级别（$O(\log n)$），所以这些操作的时间复杂度都很低。
2. **自动平衡**：在插入或删除节点时，平衡二叉树会自动进行旋转操作来保持平衡，确保性能稳定。
3. **有序性**：平衡二叉树是一种二叉搜索树，能够方便地进行有序遍历，适合用于需要排序的数据结构。

缺点

1. **复杂性**：实现平衡二叉树比普通二叉搜索树复杂，需要额外的旋转操作和高度维护。
2. **插入和删除的开销**：虽然这些操作的时间复杂度是$O(\log n)$，但每次插入或删除都可能需要多次旋转，增加了实际的操作开销。
3. **空间开销**：每个节点需要存储额外的高度信息或平衡因子，增加了空间复杂度。

29. 简述什么是红黑树?

红黑树（Red-Black Tree）是一种自平衡的二叉查找树。它在每个节点上增加了一个存储位来表示节点的颜色，可以是红色或黑色。红黑树通过一系列规则来保持树的平衡，从而确保基本操作（如查找、插入和删除）的时间复杂度为 $O(\log n)$。

红黑树的主要特性包括：

1. **节点是红色或黑色。**
2. **根节点是黑色。**
3. **所有叶子节点（空节点）都是黑色。**
4. **红色节点的子节点都是黑色**（即没有两个连续的红色节点）。
5. **从任一节点到其每个叶子节点的所有路径都包含相同数目的黑色节点。**

这些特性帮助红黑树在插入和删除节点时，通过旋转和重新着色操作来保持平衡。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

30. 红黑树适合什么样的使用场景?

红黑树适用于需要高效查找、插入和删除操作的场景，特别是在以下几种情况下表现出色：

1. **符号表和字典**：红黑树常用于实现符号表和字典，能够快速地进行键值对的查找、插入和删除操作。
2. **关联数组**：在需要动态调整大小的关联数组中，红黑树可以提供稳定的性能。
3. **操作系统中的调度器**：一些操作系统使用红黑树来管理进程调度和内存分配，因为它们需要快速插入和删除操作。
4. **数据库索引**：红黑树可以用作数据库的索引结构，帮助快速查找记录。
5. **集合和多重集合**：红黑树适合实现集合和多重集合，支持高效的元素插入、删除和查找。

红黑树的自平衡特性使得它在这些场景中能够保持较低的时间复杂度，确保操作的高效性。

31. 平衡二叉树和红黑树有什么区别?

平衡二叉树（AVL树）和红黑树都是用来保持二叉搜索树平衡的，但它们的平衡方式和规则有些不同。

平衡二叉树（AVL树）：

- 每个节点的左右子树高度差不超过1。
- 插入和删除节点时，可能需要多次旋转来保持平衡。
- 因为高度差严格控制，所以查找操作通常比较快。

红黑树：

- 每个节点要么是红色，要么是黑色。
- 根节点是黑色的。
- 红色节点的子节点必须是黑色的（不能有两个连续的红色节点）。
- 从任一节点到其每个叶子节点的路径上，黑色节点的数量相同。
- 插入和删除节点时，可能需要变色和旋转，但旋转次数通常比AVL树少。

总结一下，平衡二叉树追求的是“绝对平衡”，而红黑树追求的是“相对平衡”。红黑树的插入和删除操作通常比平衡二叉树更高效，但查找操作可能稍微慢一点。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

32. 简述什么是满二叉树?

满二叉树是一种特殊的二叉树，其中每个节点要么有两个子节点，要么没有子节点（即叶子节点）。换句话说，除了最后一层外，每一层的所有节点都有两个子节点[1][2]。

满二叉树的特点包括：

- **节点数**：如果满二叉树的深度为k，那么它的节点总数是$(2^k - 1)$。
- **叶子节点**：所有的叶子节点都在最后一层，并且数量为 2^{k-1}。
- **层级节点数**：第 i层的节点数为2^{i-1}。

这种结构使得满二叉树在同样深度的二叉树中，节点数最多。满二叉树的概念主要用于理论研究和算法分析中，它提供了一种理想化的树形结构，有助于简化某些树相关算法的设计和分析。在实际应用中，完美二叉树和完全二叉树的概念可能更加实用，特别是在涉及到二叉堆或者二叉树存储结构的设计时。

33. 简述什么是完全二叉树?

完全二叉树（Complete Binary Tree）是一种特殊的二叉树，它具有以下特性：

1. 所有层都是完全填满的，除了可能的最后一层。 在最后一层，所有的节点都尽可能地集中在左边。
2. 最后一层的节点如果不是全部填满，那么缺失的节点只能出现在层的右边，且最后一层的节点连续分布在最左边。

这意味着，如果你按层序（从上到下，从左到右）遍历完全二叉树的节点，就像在读一本书一样，你会发现没有跳过任何“页码”直到最后一“页”。如果最后一“页”不是满的，那么所有的“空位”都集中在右边。

特点

- 高效的利用空间：完全二叉树不像满二叉树那样要求每一层都完全填满，但它依然保持了较好的平衡性，因此在数组中表示时可以高效地利用空间。
- 数组表示简洁：完全二叉树可以非常容易地使用数组来表示，无需为节点间的链接使用额外的指针。对于数组中任意位置i的元素，其左子节点位置为 `2*i+1`，右子节点为 `2*i+2`，父节点为 `(i-1)/2`。

应用

完全二叉树在计算机科学中有许多重要应用，尤其是在数据结构和算法的设计中：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

1. 二叉堆：二叉堆是完全二叉树的一个典型应用，广泛用于实现优先队列。二叉堆通过维持完全二叉树的结构，使得插入和删除最大（或最小）元素的操作能够在$O(\log n)$时间内完成。
2. 树形数组和线段树：在处理某些特定的算法问题，如区间查询和更新操作时，完全二叉树的结构使得这些数据结构的实现更加高效。

与满二叉树和平衡二叉树的关系

- 满二叉树：是一种特殊的完全二叉树，其中每一层都完全填满，没有任何节点缺失。
- 平衡二叉树：如AVL树或红黑树，它们保持树的平衡以确保操作的效率，但不一定是完全二叉树。
  总的来说，完全二叉树通过其结构特性在多种场景中提供了高效的操作性能，尤其是在通过数组实现时，由于其结构的规律性，使得相关操作更加简便和高效。

34. 简述二叉树的存储方式?

二叉树的存储方式主要有两种：**顺序存储**和**链式存储**。

1. **顺序存储**：
   - 使用数组来存储二叉树的节点。
   - 对于完全二叉树，节点的存储位置可以通过数组下标直接反映出节点之间的逻辑关系。
   - 例如，根节点存储在数组的第一个位置，左孩子节点存储在第2个位置，右孩子节点存储在第3个位置，以此类推。
   - 优点是可以快速访问任意节点，但缺点是如果二叉树不完全，会浪费大量存储空间[1,2]。
2. **链式存储**：
   - 使用链表来存储二叉树的节点。
   - 每个节点包含三个域：数据域、左孩子指针和右孩子指针。
   - 左孩子指针指向左子节点，右孩子指针指向右子节点。
   - 优点是节省空间，适用于任意形态的二叉树，但访问节点时需要遍历链表[1,2]。

35. 简述什么是B-tree、B+tree多叉树?

B-tree和B+tree都是多叉树的一种，主要用于数据库和文件系统中，以提高数据存取效率。它们的主要特点和区别如下：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

B-tree（B树）

- **定义**：B树是一种自平衡的多路查找树，每个节点可以有多个子节点。
- **特点**：
  - 每个节点包含多个关键字和子节点指针。
  - 所有叶子节点都在同一层。
  - 节点的关键字按递增顺序排列，子节点的关键字范围在父节点的关键字之间。
  - 非叶子节点至少有 $\lceil M/2 \rceil$ 个子节点，最多有 (M) 个子节点（(M) 为B树的阶数）。
  - 插入和删除操作会保持树的平衡。

B+tree（B+树）

- **定义**：B+树是B树的变种，进一步优化了查询效率。
- **特点**：
  - 非叶子节点只存储关键字，不存储数据。
  - 所有数据都存储在叶子节点，叶子节点按关键字顺序链接成一个链表。
  - 非叶子节点的子节点数等于关键字数。
  - 查询操作更高效，因为所有数据都在叶子节点，且叶子节点形成有序链表，便于范围查询。

B树和B+树的比较

1. 搜索性能：B+树提供了更快的查找速度，因为所有实际数据都在叶子节点上并且叶子节点形成一个有序链表，便于快速顺序访问。
2. 空间利用率：B+树的非叶子节点不存储数据记录，仅作索引用，这样同样大小的节点可以存更多的键，使得树的高度更低，减少了磁盘I/O操作。
3. 范围查询：由于B+树的叶子节点形成了有序链表，使得B+树在进行范围查询时比B树更加高效。

36. 综合简述B 树和B+ 树的区别?

B树和B+树都是用于数据库和文件系统中的数据结构，但它们有一些关键区别：

1. **节点存储内容**：

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069

- **B树**：每个节点都存储键值和数据。也就是说，数据可以存在于非叶子节点和叶子节点中。
- **B+树**：非叶子节点只存储键值，不存储数据。所有数据都存储在叶子节点中。

2. **叶子节点的连接**：
   - **B树**：叶子节点之间没有特别的连接。
   - **B+树**：所有叶子节点通过指针连接成一个链表，这样可以更方便地进行范围查询。
3. **查询效率**：
   - **B树**：由于数据可以存在于任何节点，查询时可能会在非叶子节点找到数据，查询效率可能会稍高一些。
   - **B+树**：因为所有数据都在叶子节点，所以查询时必须遍历到叶子节点，查询效率相对固定。
4. **适用场景**：
   - **B树**：适用于对插入和删除操作要求不高的场景。
   - **B+树**：适用于需要频繁范围查询的场景，比如数据库索引。

总结一下，B+树在范围查询和磁盘IO性能上更有优势，而B树在某些情况下查询效率可能更高。

内容来源：csdn.net
作者昵称：weixin_44329069
原文链接：https://blog.csdn.net/weixin_44329069/article/details/139855398
作者主页：https://blog.csdn.net/weixin_44329069